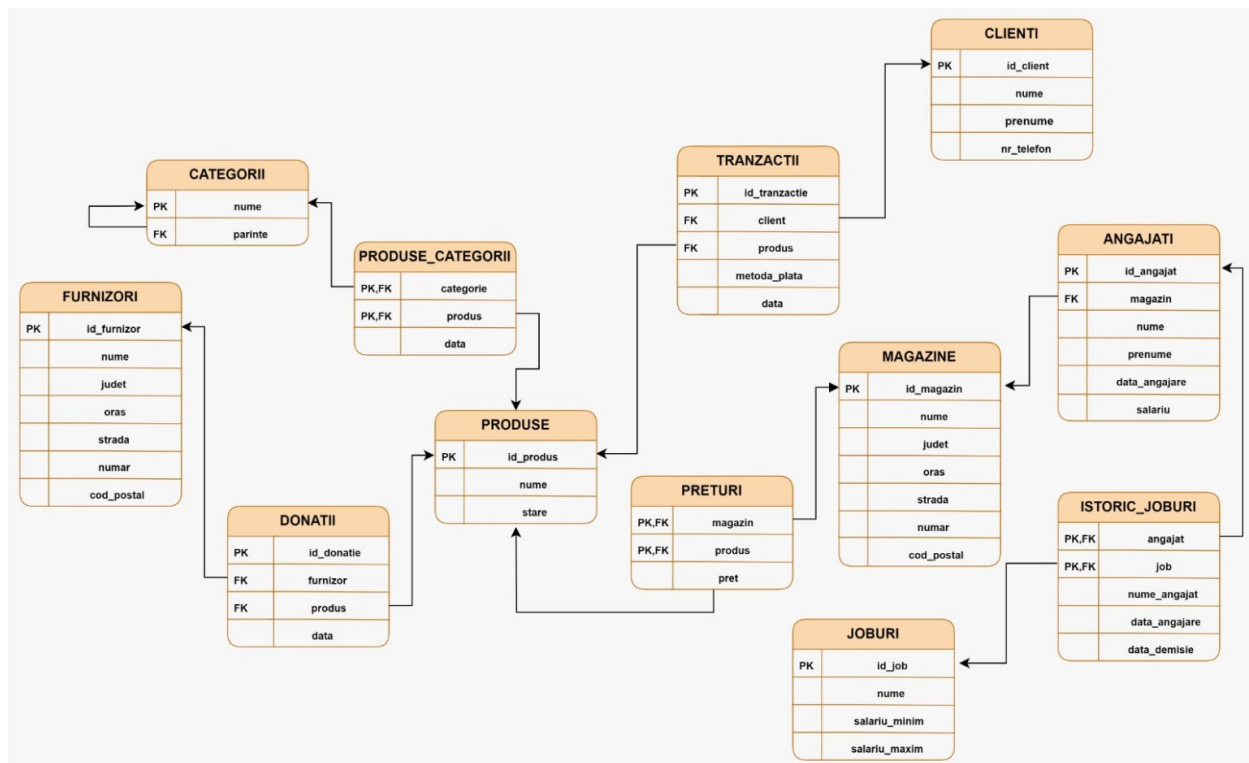
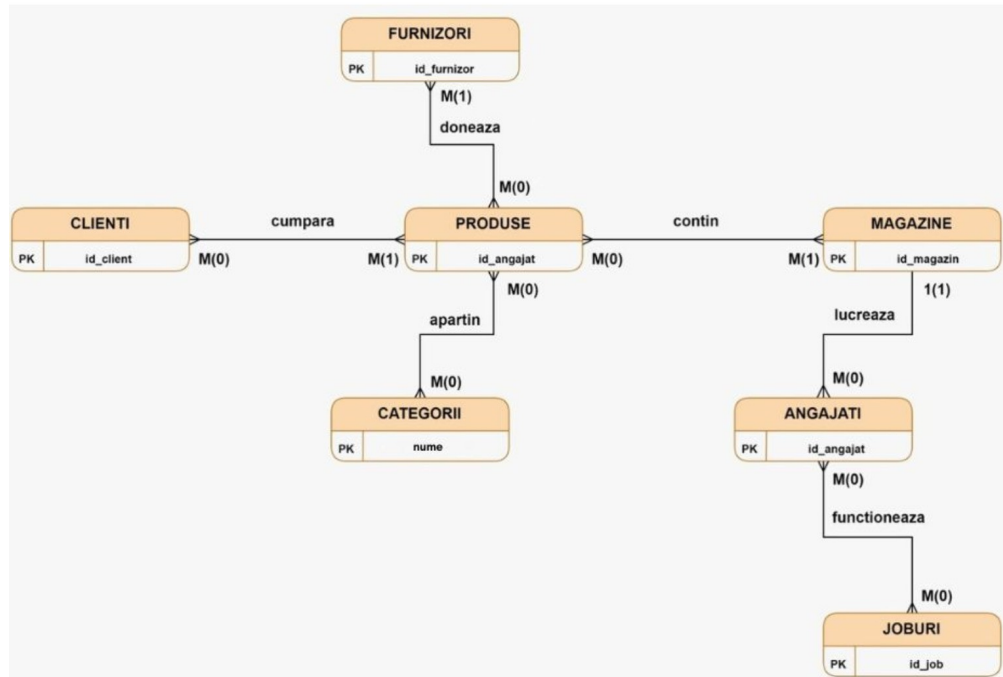


Proiect SABD



1. (1p)

a) Sa se dea un exemplu de atribut repetitiv (multivaloare) al unei entitati in modelul entitate-legatura.

Entitatea CATEGORII are un atribut repetitiv/multivaloare implicat: parinte.

Aici, un nume de categorie poate avea mai multe valori pentru atributul parinte.

Acest lucru indică faptul ca o categorie poate apartine mai multor parinti (de exemplu, subcategoriile sunt asociate cu mai multe categorii).

b) Sa se dea un exemplu de atribut repetitiv (multivaloare) al unei relatii multi-la-multi in modelul entitate-legatura.

Un atribut repetitiv (multivaloare) intr-o relatie many-to-many apare atunci cand o relatie intre doua entitati poate avea mai multe valori pentru un anumit atribut.

Relatia cumpara dintre entitățile CLIENTI și PRODUSE:

Aceasta este o relatie de tip many-to-many (M:N), deoarece un client poate cumpara mai multe produse, iar un produs poate fi cumparat de mai multi clienti.

Un atribut repetitiv (multivaloare) al acestei relatii ar putea fi:

metoda_plata: un client poate cumpara acelasi produs utilizand metode de plata diferite (cash sau card).

data: un client poate cumpara acelasi produs de mai multe ori, fiecare achizitie avand o data diferita.

c) Sa se arate cum se transforma attributele de mai sus la crearea design-ului logic al unei baze de date relationale.

```
CREATE TABLE produse
( id_produs NUMBER(3) CONSTRAINT pk_furnizori PRIMARY KEY CHECK (id_furnizor >= 0)
, nume VARCHAR2(20) NOT NULL
, stare VARCHAR2(10) DEFAULT 'Utilizat', CHECK (stare IN('Defect', 'Utilizat', 'Nou'))
, CONSTRAINT uq_produs UNIQUE (nume, stare)
);

CREATE TABLE tranzactii
( id_tranzactie NUMBER(4)
, CONSTRAINT pk_tranzactii PRIMARY KEY(id_tranzactie)
, CONSTRAINT chk_tranzactii CHECK (id_tranzactie >= 0)
, client NUMBER(4)
, CONSTRAINT fk_tranzactii_client FOREIGN KEY (client) REFERENCES clienti(id_client) ON DELETE CASCADE
, produs NUMBER(3)
, CONSTRAINT fk_tranzactii_produs FOREIGN KEY (produs) REFERENCES produse(id_produs) ON DELETE CASCADE
, metoda_plata VARCHAR2(5) DEFAULT 'Cash', CHECK (metoda_plata IN ('Cash', 'Card'))
, data DATE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE clienti
( id_client NUMBER(4)
, CONSTRAINT pk_clienti PRIMARY KEY(id_client)
, CONSTRAINT chk_clienti CHECK (id_client >= 0)
, nume VARCHAR2(20) NOT NULL
, prenume VARCHAR2(20) NOT NULL
, nr_telefon VARCHAR2(10) NOT NULL CHECK (REGEXP_LIKE(nr_telefon, '^([0-9]{10})$') AND SUBSTR(nr_telefon, 1, 2) = '07')
, CONSTRAINT uq_clienti UNIQUE (nume, prenume, nr_telefon)
);
```

2.(0.5p)

a) Sa se dea un exemplu de relatie de tip 3 (intre mai mult de doua entitati) in modelul entitate-legatura.

Relatia de tip 3 implica mai mult de doua entitati. Aceste relații sunt mai rare în proiectarea bazelor de date, dar apar atunci cand exista o dependenta logica untre trei sau mai multe entitati.

Exemplu de relatie de tip 3:

In baza ta de date, putem considera o relatie intre PRODUSE, FURNIZORI și DONATII. Relatia dintre acestea poate fi descrisă astfel:

FURNIZORI: Entitatea care face donatia.

PRODUSE: Entitatea care reprezinta obiectul donat.

DONATII: Relatia care conecteaza furnizorii si produsele.

Relatie intre cele trei entitati:

Un furnizor poate dona mai multe produse.

Un produs poate fi donat de mai multi furnizori.

Fiecare donatie este inregistrata cu o data (atribut specific relatiei).

```
CREATE TABLE donatii
( id_donatie NUMBER(4) CONSTRAINT pk_donatii PRIMARY KEY CHECK (id_donatie >= 0)
, furnizor NUMBER(2)
, produs NUMBER(2)
, data DATE DEFAULT CURRENT_TIMESTAMP
, CONSTRAINT fk_donatii_furnizor FOREIGN KEY (furnizor) REFERENCES furnizori(id_furnizor) ON DELETE CASCADE
, CONSTRAINT fk_donatii_produs FOREIGN KEY (produs) REFERENCES produse(id_produs) ON DELETE CASCADE
);
```

b) Sa se dea un exemplu de trei sau mai multe entitati care nu formeaza o relatie de tip 3, ci, relatia aparenta de tip 3, „se sparge” de fapt in relatii multi-la-multi (intre cate doua entitati).

Exemplu: PRODUSE, CATEGORII si MAGAZINE

Entitati implicate:

PRODUSE: Reprezinta produsele disponibile.

CATEGORII: Reprezinta categoriile in care produsele sunt clasificate.

MAGAZINE: Reprezinta locatiile unde produsele sunt disponibile.

Aceasta relatie se descompune in doua relatii many-to-many, deoarece legatura dintre aceste entitati poate fi tratata separat:

Descompunerea relatiei:

Relatia dintre PRODUSE si CATEGORII:

Un produs poate apartine mai multor categorii.

O categorie poate contine mai multe produse.

Ele sunt legate prin tabelul intermediar PRODUSE_CATEGORII

Relatia dintre PRODUSE si MAGAZINE:

- Un produs poate fi vandut in mai multe magazine.
- Un magazin poate vinde mai multe produse.
- Ele sunt legate prin tabelul intermediar PRETURI.

4.(0.5p)

Sa se dea un exemplu de tabel relational in care exista o dependenta multivaloare (multidependenta) intre atributele (coloanele) sale, care nu este dependenta functionala.

Relatia dintre furnizor si produs este una de multidependenta, deoarece:

- Un furnizor poate dona mai multe produse.
- Un produs poate fi donat de mai multi furnizori.
- Valorile pentru produs sunt independente de valorile pentru furnizor atunci cand ne raportam doar la pk id_donatie.

Nu este o dependenta funcțională:

- produs nu este determinat de furnizor, și vice-versa, atunci cand privim doar pk.

In tabela DONATII, furnizor și produs sunt in multidependenta:

- Donatiile unui furnizor pot include mai multe produse.
- Produsele pot proveni de la mai multi furnizori.

```
CREATE TABLE donatii
( id_donatie NUMBER(4) CONSTRAINT pk_donatii PRIMARY KEY CHECK (id_donatie >= 0)
, furnizor NUMBER(2)
, produs NUMBER(2)
, data DATE DEFAULT CURRENT_TIMESTAMP
, CONSTRAINT fk_donatii_furnizor FOREIGN KEY (furnizor) REFERENCES furnizori(id_furnizor) ON DELETE CASCADE
, CONSTRAINT fk_donatii_produs FOREIGN KEY (produs) REFERENCES produse(id_produs) ON DELETE CASCADE
);
```

```
INSERT INTO donatii(id_donatie, furnizor, produs, data) VALUES (0, 0, 0, TO_DATE('2023-06-12', 'YYYY-MM-DD'));
INSERT INTO donatii(id_donatie, furnizor, produs, data) VALUES (1, 0, 3, TO_DATE('2023-02-1', 'YYYY-MM-DD'));
INSERT INTO donatii(id_donatie, furnizor, produs, data) VALUES (2, 1, 1, TO_DATE('2021-04-8', 'YYYY-MM-DD'));
INSERT INTO donatii(id_donatie, furnizor, produs, data) VALUES (3, 2, 0, TO_DATE('2022-03-1', 'YYYY-MM-DD'));
INSERT INTO donatii(id_donatie, furnizor, produs, data) VALUES (4, 2, 3, TO_DATE('2022-04-26', 'YYYY-MM-DD'));
INSERT INTO donatii(id_donatie, furnizor, produs, data) VALUES (5, 4, 4, TO_DATE('2023-11-4', 'YYYY-MM-DD'));
INSERT INTO donatii(id_donatie, furnizor, produs) VALUES (6, 4, 2);
INSERT INTO donatii(id_donatie, furnizor, produs) VALUES (7, 4, 2);
INSERT INTO donatii(id_donatie, furnizor, produs, data) VALUES (8, 3, 2, TO_DATE('2020-01-19', 'YYYY-MM-DD'));
```

5. (0.5p) Sa se ilustreze printr-un exemplu structura unui index de tip arbore B* si modul in care o interogare SQL foloseste acest index.

```
INSERT INTO produse(id_produș, nume, stare) VALUES(0, 'Laptop cu SSD', 'Nou');
INSERT INTO produse(id_produș, nume, stare) VALUES(1, 'PC i7 ', 'Utilizat');
INSERT INTO produse(id_produș, nume, stare) VALUES(2, 'Tableta Android', 'Defect');
INSERT INTO produse(id_produș, nume, stare) VALUES(3, 'Gaming Laptop RGB', 'Nou');
INSERT INTO produse(id_produș, nume, stare) VALUES(4, 'Aspirator Cyclone', 'Utilizat');
```

Pe acest set de date putem ordona lexicografic numele produsului. Vom avea radacina [Laptop cu SSD] si cele noduri frunza [Aspirator Cyclone, Gaming Laptop RGB] si [PC i7, Tableta Android]. Valorile mai mici ca radacina vor merge in stanga iar cele mai mari spre dreapta.

```
CREATE INDEX idx_nume_produce
ON produse (nume);
```

```
-----
SELECT *
FROM produse
WHERE nume = 'Gaming Laptop RGB';
```

6. (0.5p) Sa se ilustreze printr-un exemplu structura unui index de tip bitmap si modul in care o interogare SQL foloseste acest index.

Pe acelasi set de date putem crea un index bitmap pe coloana stare. Vom avea urmatoarele valori Nou = [1, 0, 0, 1, 0], Utilizat = [0, 1, 0, 0, 1] si Defect = [0, 0, 1, 0, 0].

```
CREATE BITMAP INDEX idx_stare_produce
ON produse (stare);
```

```
-----
SELECT *
FROM produse
WHERE stare IN ('Nou', 'Defect');
```

Vor rezulta toate produsele fara cele cu starea "Utilizat".

7. (0,5p) Sa se dea un exemplu de vedere (vizualizare) VIEW_EX astfel incat comanda INSERT INTO VIEW_EX VALUES sa produca o eroare. Sa se explice cauza erorii.

Utilizam datele din tabelele Produse, Categori si Produse_Categorii.

```
CREATE VIEW VIEW_EX AS
SELECT p.id_produș, p.nume AS nume_produș, c.nume AS categorie
FROM produse p
JOIN produse_categorii pc ON p.id_produș = pc.produș
JOIN categorii c ON pc.categorie = c.nume;
```

Daca vom incerca sa inseram date in acest view deoarece cererea este ambigua pentru sistemul SQL si nu va stii in ce tabel sa insereze aceste date.

8. (0,5p) Sa se ilustreze printr-un exemplu modul in care o vedere (vizualizare) poate fi folosita pentru a asigura securitatea intr-o baza de date.

Vom face un view pe tabelul clienti unde vom ascunde numarul de telefon si numele persoanei.

```
CREATE VIEW clienti_view AS  
SELECT id_client, prenume  
FROM clienti;
```

9. (1p) Sa se arate printr-un exemplu in ce conditii este posibil ca doua select-uri identice consecutive (fara nici o alta comanda intre ele), efectuate in aceeasi sesiune de lucru, pe acelasi tabel, pot produce rezultate diferite.

(date la tranzactii dar asta presupune inserarea unei noi tranzactii)

```
SELECT *  
FROM FURNIZORI  
ORDER BY DBMS_RANDOM.RANDOM
```

10. (0,5p) Sa se dea un exemplu care sa ilustreze interblocarea.

```
BEGIN;  
UPDATE tranzactii SET metoda_plata = 'Card' WHERE id_tranzactie = 1;  
  
BEGIN;  
UPDATE clienti SET nume = 'Popa' WHERE id_client = 2;  
  
UPDATE clienti SET prenume = 'Maria' WHERE id_client = 2;  
  
UPDATE tranzactii SET metoda_plata = 'Cash'  
WHERE id_tranzactie = 1;
```

Tranzactia 1 asteapta ca tranzactia 2 sa elibereze randul din tabelul clienti, iar tranzactia 2 asteapta ca tranzactia 1 sa elibereze randul din tabelul tranzactii.

11. (0.5p) Sa se ilustreze printr-un exemplu utilizarea unui trigger pentru a realiza o constrangere de integritate care nu ar putea fi implementata folosind un CONSTRAINT din definitia unui tabel.

```
CREATE TRIGGER trigger_oras_pitesti  
BEFORE INSERT OR UPDATE ON furnizori  
FOR EACH ROW  
BEGIN  
    IF :NEW.oras != 'Pitesti' THEN  
        RAISE_APPLICATION_ERROR(-20002, 'Orasul trebuie sa fie "Pitesti".');  
    END IF;  
END;
```

13. (1p) Sa se ilustreze prin exemple folosirea instructiunii RAISE pentru a ridica atat o exceptie predefinita cat si o exceptie definita de utilizator. In cazul exceptiilor predefinite, sa se explice cum anume folosirea instructiunii RAISE schimba functionalitatea programului (fata de cazul cand aceasta instructiune nu exista).

DECLARE

v_nume_furnizor VARCHAR2(20);

BEGIN

DBMS_OUTPUT.PUT_LINE('search furnizor...');

**SELECT nume
INTO v_nume_furnizor
FROM furnizori
WHERE id_furnizor = 99;**

DBMS_OUTPUT.PUT_LINE('furnizor: ' || v_nume_furnizor);

EXCEPTION

WHEN NO_DATA_FOUND THEN

DBMS_OUTPUT.PUT_LINE('Eroare: furnizorul cu id_furnizor = 99 nu exista.');

END;

DECLARE

e_cod_postal_invalid EXCEPTION; -- exceptia

v_cod_postal VARCHAR2(6) := '67231';

BEGIN

DBMS_OUTPUT.PUT_LINE('check cod postal...');

IF LENGTH(v_cod_postal) != 6 OR NOT REGEXP_LIKE(v_cod_postal, '^[0-9]{6}\$') THEN

RAISE e_cod_postal_invalid; -- ridicare exceptie

END IF;

DBMS_OUTPUT.PUT_LINE('Codul postal este valid.');

EXCEPTION

WHEN e_cod_postal_invalid THEN

DBMS_OUTPUT.PUT_LINE('Eroare: Codul postal "' || v_cod_postal || '" este invalid.');

END;

3.(1p) a) Sa se dea un exemplu de tabel relational care este in FN1, dar nu in FN2. Sa se aduca tabelul in FN2.

produs	categorie	data
0	Desktop	2024-01-01
1	Electronice	2022-12-1
2	Aspiratoare	2024-10-22
3	Tablete	2023-01-23

In tabelul Produse_Categorii avem 3 coloane produs, categorie si data unde pk este format din (produs, categorie). Grupam produsul cu data si produsul cu categorie pentru a forma tabelul relational de tip FN2.

<u>produs</u>	<u>categorie</u>
0	Desktop
1	<u>Electronice</u>
2	Aspiratoare
3	<u>Tablete</u>

b) Sa se dea un exemplu de tabel relational care este in FN2, dar nu in FN3. Sa se aduca tabelul in FN3.

Daca am adauga coloana Judet in tabelul Angajati, acest atribut va depinde de magazin, care la randul sau depinde de id_angajat. Grupam coloana magazin cu judetul, si vom ramane cu tabelul format din id_angajat, nume, prenume, data_angajare si salariu.

id_angajat	nume	prenume	data_angajare	salariu
1	Marian	Mihai	2021-02-30	2400
2	Olud	Ionut	2023-01-09	2881
3	Tudor	Vlad	2012-12-02	3000